

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Database Design and Connectivity

COURSE NAME: Query Processing for Data Science **COURSE CODE:** DSA0503

COURSE OUTCOME COVERED

CO2: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)

Assessment Tool Weightage: 35%

Dave's Taxonomy: Manipulation (Level 2)

TotalMarks:20

Question Count: 2

Code of Conduct

I ____P. Surya(192324222)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____surya_____

Database Design and Connectivity Rubric

Requirement Analysis (4 Marks)	ER Diagram Design (4 Marks)	Table Design & Normalization (4 Marks)	Database Connectivity using Python (4 Marks)	CRUD Operations (4 Marks)	Total (20 Marks)

--	--	--	--	--	--

Total Marks Awarded:

Questions:

1.Examination Result Management System (10 Marks)

A college wants to store student details and examination results in a MySQL database. Design Student and Result

tables with appropriate primary and foreign keys. Write a Python program to connect to the database, create the tables, insert records, calculate the total marks of each student

Python Code

```
import mysql.connector
# Connect to MySQL
con = mysql.connector.connect(
host="localhost",
user="root",
password="root",
database="college_db"
)
cur = con.cursor()
# Create tables
cur.execute("""
CREATE TABLE IF NOT EXISTS Student (
student_id INT PRIMARY KEY,
student_name VARCHAR(50) NOT NULL,
department VARCHAR(50)
)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS Result (
result_id INT PRIMARY KEY,
student_id INT,
subject1 INT,
subject2 INT,
subject3 INT,
FOREIGN KEY (student_id) REFERENCES Student(student_id)
)
""")
```

```

)
)
# Insert student records
students = [
(101, "Surya", "AI & DS"),
(102, "Rahul", "CSE"),
(103, "Priya", "ECE")
]
cur.executemany(
"INSERT IGNORE INTO Student VALUES (%s, %s, %s)", students
)
# Insert result records
results = [
(1, 101, 85, 90, 88),
(2, 102, 78, 82, 80),
(3, 103, 92, 89, 95)
]
cur.executemany(
"INSERT IGNORE INTO Result VALUES (%s, %s, %s, %s, %s)", results
)
con.commit()
# Calculate total marks using SQL JOIN
query = """
SELECT s.student_id, s.student_name,
r.subject1, r.subject2, r.subject3,
(r.subject1 + r.subject2 + r.subject3) AS total
FROM Student s
JOIN Result r ON s.student_id = r.student_id
ORDER BY s.student_id
"""
cur.execute(query)
print("EXAMINATION RESULTS")
print("-" * 70)
print("ID Name Sub1 Sub2 Sub3 Total")

print("-" * 70)
for row in cur.fetchall():
print(f'{row[0]:<4} {row[1]:<10} {row[2]:<5} {row[3]:<5} '
f'{row[4]:<5} {row[5]}')
cur.close()
con.close()

```

Output:

EXAMINATION RESULTS

ID	Name	Sub1	Sub2	Sub3	Total
101	Surya	85	90	88	263
102	Rahul	78	82	80	240
103	Priya	92	89	95	276

2. Airline Reservation System (10 Marks)

An airline company wants to maintain passenger information and flight booking details in a PostgreSQL database.

Design Passenger and Booking tables with suitable primary and foreign keys. Write a Python program to connect to the PostgreSQL database, create the tables, insert booking records, update the booking status of a passenger, and display passenger names along with their corresponding flight details using an SQL JOIN query.

Python Code:

```
import psycopg2
# Connect to PostgreSQL
con = psycopg2.connect(
    host="localhost",
    database="airline_db",
    user="postgres",
```

```

password="postgres"
)
cur = con.cursor()
# Create tables
cur.execute("""
CREATE TABLE IF NOT EXISTS Passenger (
passenger_id INT PRIMARY KEY,
passenger_name VARCHAR(50) NOT NULL,
phone VARCHAR(15)
)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS Booking (
booking_id INT PRIMARY KEY,
passenger_id INT,
flight_no VARCHAR(20),
flight_date DATE,
booking_status VARCHAR(20),
FOREIGN KEY (passenger_id) REFERENCES Passenger(passenger_id)
)
""")
# Insert passenger records
passengers = [
(201, "Arun", "9876543210"),
(202, "Meena", "9876501234"),
(203, "Kiran", "9876512345")
]
cur.executemany("""
INSERT INTO Passenger (passenger_id, passenger_name, phone)
VALUES (%s, %s, %s)
ON CONFLICT (passenger_id) DO NOTHING
""", passengers)
# Insert booking records
bookings = [
(1, 201, "AI101", "2026-08-10", "Confirmed"),
(2, 202, "6E205", "2026-08-11", "Confirmed"),
(3, 203, "UK301", "2026-08-12", "Confirmed")
]
cur.executemany("""
INSERT INTO Booking

```

```

(booking_id, passenger_id, flight_no, flight_date, booking_status)
VALUES (%s, %s, %s, %s, %s)
ON CONFLICT (booking_id) DO NOTHING
""" , bookings)
con.commit()
# Update booking status
cur.execute("""
UPDATE Booking
SET booking_status = 'Cancelled'
WHERE passenger_id = 202
""")
con.commit()
# Display passenger and flight details using JOIN
query = """
SELECT p.passenger_name, b.flight_no,
b.flight_date, b.booking_status
FROM Passenger p
JOIN Booking b ON p.passenger_id = b.passenger_id
ORDER BY p.passenger_id
"""
cur.execute(query)
print("AIRLINE RESERVATION DETAILS")
print("-" * 65)
print("Passenger Flight Date Status")
print("-" * 65)
for row in cur.fetchall():
print(f'{row[0]:<11} {row[1]:<8} {row[2]} {row[3]}')
cur.close()
con.close()

```

Output:

AIRLINE RESERVATION DETAILS

Passenger	Flight	Date	Status
Arun	AI101	2026-08-10	Confirmed
Meena	6E205	2026-08-11	Cancelled
Kiran	UK301	2026-08-12	Confirmed

Faculty

Signature of the Course